

<p> Chapter 3 - Client-Side Applications </p>

Introduction to ReactJS

<p> Er. Ramesh Sharma </p>



Client-Side Applications

- A **client-side** application runs **mainly in the** user's browser.
The browser handles:
 - • UI rendering
 - User interaction
 - Data displayThe **server** mainly provides data (APIs), not full HTML pages.
- Examples: **Gmail, Facebook, YouTube, Online dashboards**





Traditional Multi-Page Application (MPA)

- A web application where each request loads a completely new HTML page from the server.

How It Works

- User clicks a link
- Browser sends request to server
- Server returns a new HTML page
- Browser reloads the entire page





Single Page Application [SPA]

- A web application that loads a single HTML page once and dynamically updates content using JavaScript.

How It Works

- First load → HTML + CSS + JS loaded
- User interaction → JavaScript updates content
- Data fetched via APIs (no full reload)





What is React JS?

- React JS is a JavaScript library used to build client-side user interfaces, especially Single Page Applications (SPAs).
- Developed by Meta (Facebook)
- Focuses only on UI (View layer)





What is React JS?

- Latest version of React.JS is 19.0.0 (December 2024).

Initial release to the Public (version 0.3.0) was in July 2013.

- React.JS was first used in 2011 for Facebook's Newsfeed feature.
- Facebook Software Engineer, Jordan Walke, created it.





What is React JS?

- React is a JavaScript library for rendering user interfaces (UI).
- UI is built from small units like buttons, text, and images.
- React lets you combine them into reusable, nestable components.
- From web sites to phone apps, everything on the screen can be broken down into components.





JSX (JavaScript XML)

- JSX allows writing HTML inside JavaScript.

```
const heading = <h1>Hello React</h1>;
```

Easier to read.

Combines UI + logic.





JSX (JavaScript XML)

- Each React component is a JavaScript function that may contain some markup that React renders into the browser.

React components use a syntax extension called JSX to represent that markup.





Getting Started

#download node.js

<https://nodejs.org/en/download>

```
npm install -g npm@11.9.0
```

#Install a Build Tool (Vite)

```
npm install -g create-vite
```

#Create a React Application

```
npm create vite@latest my-react-app -- --template react
```





Getting Started

- #path to root folder
cd my-react-app

#Install Dependencies
npm install

#Run
npm run dev





React Render HTML

- React uses a container to render HTML in a web page.

Typically, this container is a `<div id="root"></div>` element in the `index.html` file.

The `createRoot` function is located in the `main.jsx` file in the `src` folder, and is a built-in function that is used to create a root node for a React application.

The `createRoot()` function takes one argument, an HTML element.





The render Method

The render method defines what to render in the HTML container.

Example: main.jsx

```
import { createRoot } from 'react-dom/client';  
createRoot(document.getElementById('root')).render(<p>Welcome!</p>);
```





The render Method

```
import { createRoot } from 'react-dom/client';
```

```
const myelement = (<table><tr><th>Name</th></tr><tr><td>John</td></tr><tr><td>Elsa</td></tr></table>);
```

```
createRoot(document.getElementById('root')).render(  
  myelement  
);
```





What is JSX?

- JSX stands for JavaScript XML.
- JSX allows us to write HTML in React.
- JSX makes it easier to write and add HTML in React.

```
const myElement = <h1>I Love JSX!
```

```
</h1>;createRoot(document.getElementById('root')).render(  
  myElement  
)
```





Expressions in JSX

With JSX you can write expressions inside curly braces { }.

```
const myElement = <h1>React is {5 + 5} times better with JSX</h1>;
```

```
function Car() {  
  const hp = 218 * 1.36;  
  return (<  
    <h1 className="my-class">My car</h1>  
    <p>It has {hp} horsepower</p>  
  </>)  
};
```





Event Attributes in JSX

```
function Car() {  
  
  const myfunc = () => {alert('Hello World');}  
  
  return (<button onClick={myfunc}>Click me</button>);  
  
}
```

Event attributes in JSX are written in **camelCase**.





Style Attributes in JSX

```
function Car() {  
  const mystyles = {color: "red",fontSize: "20px",};  
  
  return (  
    <>  
    <h1 style={mystyles}>My car</h1>  
    </>);  
  }
```





JSX If Statements

```
function Fruit() {  
  const x = 5;  
  
  return (  
    <h1>{(x) < 10 ? "Banana" : "Apple"}</h1>);  
  }  
}
```





Components

React **applications** are built from **isolated pieces** of UI called components.

A React component is a JavaScript function that you can sprinkle with markup.

Components can be as small as a button, or as large as an entire page.

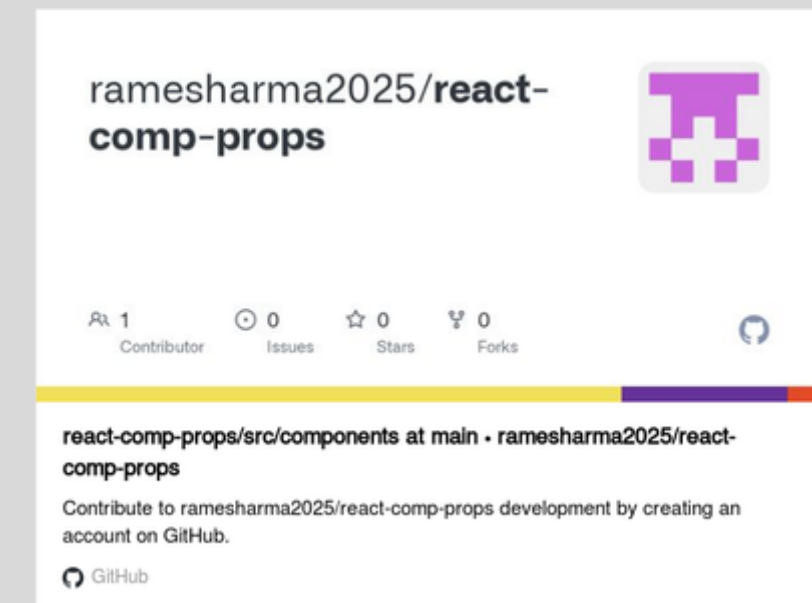
Components come in two types, Class components and Function components





Components: Example

```
● export default function Profile() {  
  return (  
      
  );  
}
```





Components: Example

- `function Car() {return (<h2>I am a Car!</h2>);}`

```
function Garage() {  
  return (  
    <><h1>Who lives in my Garage?</h1>  
    <Car /></>  
  );  
}
```





React Props

- Props are arguments passed into React components.

Props are passed to components via HTML attributes.

props stands for properties.

React Props are like function arguments in JavaScript and attributes in HTML.





React Props

- To **send props** into a component, **use the same syntax as HTML attributes**:

```
function Car(props) {  
  return <h2>I am a {props.brand}!</h2>;  
}  
  
function Garage() {  
  return (  
    <>  
      <h1>Who lives in my Garage?</h1>  
      <Car brand="Ford" />  
      <Car brand="BMW" />  
    </>  
  );  
}
```



React Props

```
1 function Car(props) {
2   return (
3     <h2>
4       I am a {props.color} {props.brand} {props.model}!
5     </h2>
6   );
7 }
8
9 function Garage() {
10  return (
11    <>
12    <Car brand="Ford" model="Mustang" color="red" />
13    </>
14  );
15 }
```



React Props - Object

```
function Car(props) {  
  return (  
    <>  
      <h2>  
        My {props.carinfo.name} {props.carinfo.model}!  
      </h2>  
    </>  
  );  
}  
  
const carInfo = {  
  name: "Ford",  
  model: "Mustang",  
  color: "red",  
  year: 1969,  
};
```

```
function Garage() {  
  return (  
    <>  
      <Car carinfo={carInfo} />{" "  
    </>  
  );  
}
```



React Props - Array

```
function Car(props) {  
  return (  
    <h2>  
      My car is a {props.carinfo[0]} {props.carinfo[1]}!  
    </h2>  
  );  
}  
  
const carInfo = ["Ford", "Mustang"];  
  
function Garage() {  
  return (  
    <>  
      <Car carinfo={carInfo} />  
    </>  
  );  
}
```



Destructuring Props

```
function Car(props) {  
  const { brand, model } = props;  
  return (  
    <h2>  
      I love my {brand} {model}!  
    </h2>  
  );  
}
```

```
function Garage() {  
  return (  
    <>  
    <Car brand="Ford" model="Mustang" color="red" year={1969} />  
    </>  
  );  
}
```



Destructuring Props: rest operator

```
function Car({ color, brand, ...rest }) {  
  return (  
    <h2>  
      My {brand} {rest.model} is {color}!  
    </h2>  
  );  
}
```

```
function Garage() {  
  return (  
    <>  
      <Car brand="Ford" model="Mustang" color="red" year={1969} />  
    </>  
  );  
}
```



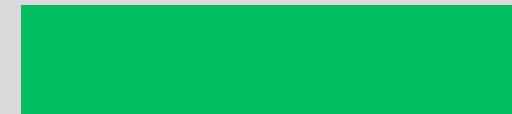
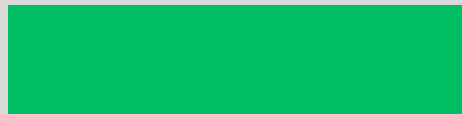
Props: default

```
function Car({ color = "blue", brand }) {  
  return (  
    <h2>  
      My {color} {brand}!  
    </h2>  
  );  
}
```

```
function Garage() {  
  return (  
    <>  
      <Car brand="Ford" model="Mustang" color="red" year={1969} />  
    </>  
  );  
}
```



Props Children



In React, you can send the content between the **opening and closing tags** of a component, to another component.

This can be **accessed in the other component** using the **props.children** property.





Props Children

```
function Car(props) {  
  return (  
    <>  
    <h2>{props.children}</h2>  
    </>  
  );  
}  
  
function Garage() {  
  return (  
    <>  
    <Car>  
      <h1>Hello World</h1>  
    </Car>  
    </>  
  );  
}
```



React Events

- Just like HTML DOM events, React can perform actions based on user events.

React events are written in **camelCase** syntax:
onClick instead of onclick.

React **event handlers** are written inside **curly braces**:

onClick={shoot} instead of onclick="shoot()"





React Events

```
function Car(props) {  
  const myFunc = () => {  
    alert("Clicked");  
  };  
  return (  
    <>  
      <div>{props.children}</div>  
      <button onClick={myFunc}>Click here</button>  
    </>  
  );  
}
```



React Events - with arguments passed

```
function Car(props) {  
  const myFunc = (text) => {  
    alert(text);  
  };  
  return (  
    <>  
      <div>{props.children}</div>  
      <button onClick={() => myFunc("clicked once!")}>Click here</button>  
    </>  
  );  
}
```





React Lists with map()

```
function MyCars() {  
  const cars = ["Ford", "BMW", "Audi"];  
  return (  
    <>  
      <h1>My Cars:</h1>  
      <ul>  
        {cars.map((car) => (  
          <li>I am a {car}</li>  
        ))}  
      </ul>  
    </>  
  );  
}
```



Keys in React Lists

```
function MyCars() {  
  const cars = [  
    { id: 1001, brand: "Ford" },  
    { id: 1002, brand: "BMW" },  
    { id: 1003, brand: "Audi" },  
  ];  
  return (  
    <>  
      <h1>My Cars:</h1>  
      <ul>  
        {cars.map((car) => (  
          <li key={car.id}>I am a {car.brand}</li>  
        ))}  
      </ul>  
    </>  
  );  
}
```

Keys allow React to keep track of elements. This way, if an item is updated or removed, only that item will be re-rendered instead of the entire list.



Array Index as Keys

```
function MyCars() {  
  const cars = ["Ford", "BMW", "Audi"];  
  return (  
    <>  
    <h1>My Cars:</h1>  
    <ul>  
      {cars.map((car, index) => (  
        <li key={index}>I am a {car}</li>  
      ))}  
    </ul>  
    </>  
  );  
}
```



React Forms

```
function MyForm() {  
  const [name, setName] = useState("");  
  
  function handleChange(e) {  
    setName(e.target.value);  
  }  
  
  return (  
    <form>  
      <label>  
        Enter your name:  
        <input type="text" value={name} onChange={handleChange} />  
      </label>  
      <p>Current value: {name}</p>  
    </form>  
  );  
}
```

The value of the input element is driven by the React state, and any changes to that value are managed through event handlers that update the state.



React Forms Submit

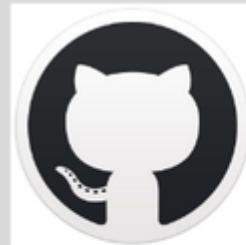
```
function MyForm() {  
  const [name, setName] = useState("");  
  
  function handleChange(e) {  
    setName(e.target.value);  
  }  
  
  return (  
    <form>  
      <label>  
        Enter your name:  
        <input type="text" value={name} onChange={handleChange} />  
      </label>  
      <p>Current value: {name}</p>  
    </form>  
  );  
}
```





Class Work

Counter with Increment, Decrement and Reset



react-comp-props/src/components/Counter.jsx at
main · ramesharma2025/react-comp-props

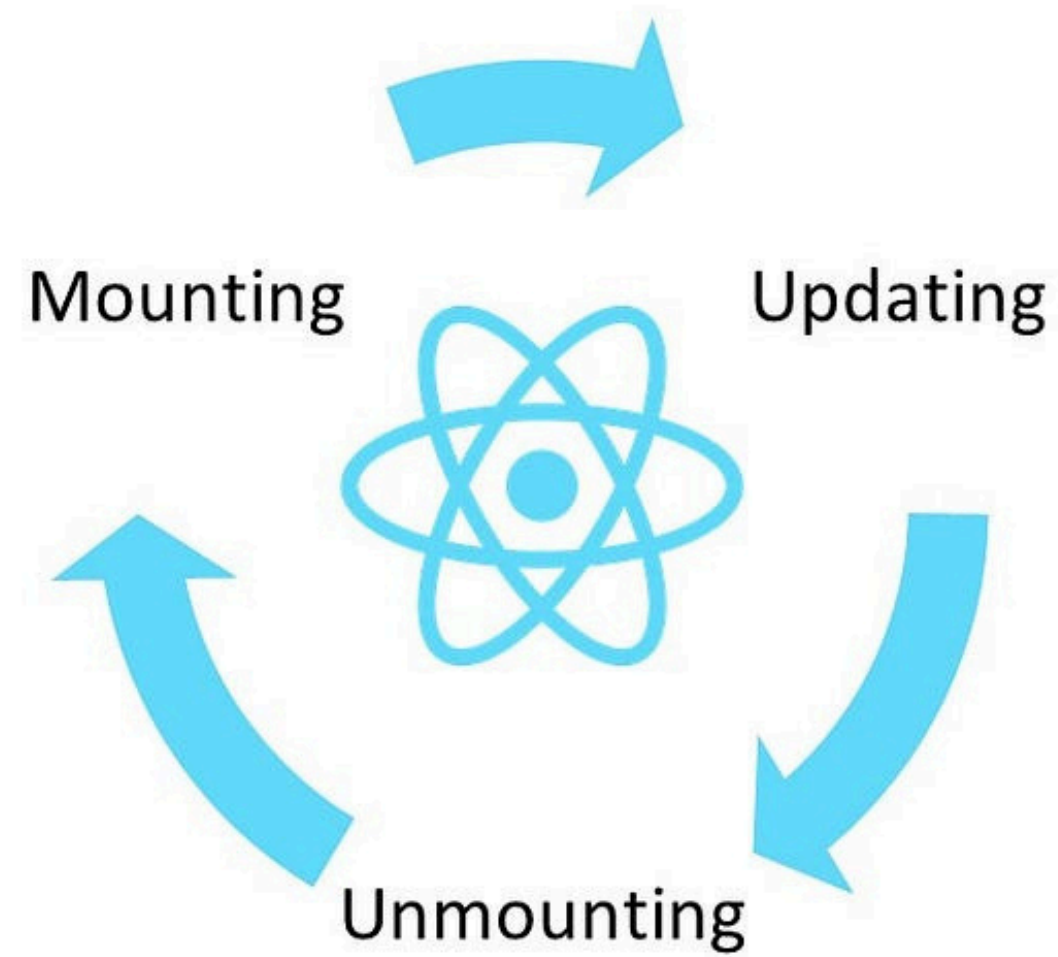
Contribute to ramesharma2025/react-comp-props
development by creating an account on GitHub.

 GitHub



React Component Lifecycle

React Component Lifecycle





React Component Lifecycle

- The React component lifecycle describes the different stages a component goes through, allowing code to run at specific moments during its existence.
 - Covers phases like creation, update, and removal of a component.
 - Helps manage side effects and resources efficiently.
 - Improves application performance through controlled execution.





State Management & Data Flow in React

State is a built-in object used to **store dynamic data** in a component.

When **state changes**, React **re-renders** the component automatically.

State is **local and private to the component** where it is defined.

Ramesh Sharma

Examples of state:

- Form input values
- **Counter values**
- Toggle (show/hide) status
- **API response data**

Ramesh Sharma





What is State Management?

State management is the process of:

- Storing application data
- Updating data correctly
- Sharing data between components

Ramesh Sharma

Types:

1. **Local State** → useState
2. **Shared State** → parent component
3. **Global State** → **Redux, Context API**
(for large apps)

Ramesh Sharma



One-Way Data Flow (Unidirectional Flow)

- React follows one-way data flow:
Parent → Child
- Data flows **downwards** using props
- **Child cannot modify parent state** directly
- This makes **applications predictable and easier to debug**





Parent-Child State Management

Case 1: State in Parent, Data to Child

- Parent holds state
- Child receives data via props

Ramesh Sharma

```
function Parent() {  
  const [age, setAge] =  
    useState(20);  
  return <Child age={age} />;  
}
```

Ramesh Sharma



Parent-Child State Management

Case 2: Child Updates Parent State (Lifting State Up)

- Parent passes **state updater function to child**
- **Child triggers** changes using **callback**

```
function Parent() {  
  const [count, setCount] = useState(0);  
  return <Child increment={setCount} count=  
    {count} />;  
}  
  
function Child({ increment, count }) {  
  return (  
    <button onClick={() => increment(count + 1)}>  
      Increment  
    </button>  
  );  
}
```



What are Hooks?

Hooks are **special functions** that let you:

- **Use state**
- **Use lifecycle features**
- **Reuse logic**

in **functional components**.



Why Hooks are Important?

- No need for class components
- Cleaner and readable code
- Reusable logic
- Better separation of concerns



Types of Hooks

1. **useState**

- Manages local component state

2. **useEffect**

- Handles side effects
- API calls, subscriptions, timers

3. **useContext**

- Shares data without prop drilling

4. **useRef**

- Access DOM elements
- Store mutable values without re-render

5. **useReducer**

- Alternative to useState for complex state logic

6. **Custom Hooks**



useState Hook

- **useState** is a **Hook** used to **add state** to functional components.

Syntax

- `const [state, setState] = useState(initialValue);`



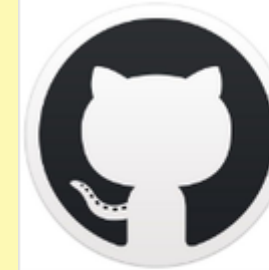


Custom Hook

A **custom hook** is a **JavaScript** function that:

- Starts with **use**
- Uses built-in hooks inside it (useState, useEffect, etc.)
- Allows **reusing** logic across components
- Return data and functions

```
function useCustomHookName() {  
  // useState, useEffect, logic  
  return something;  
}
```





Client Side Routing- React Router

Routing allows **switching** between **different views/pages** without **reloading** the browser. **npm install react-router-dom**

In traditional websites:

- Clicking a link → Full page reload.

In Single Page Applications (SPA):

- Content updates dynamically.
- No full page refresh.
- Faster & smoother UX.





BrowserRouter

- Wraps the application.

Example:

```
import { BrowserRouter } from "react-router-dom";
```

```
<BrowserRouter>
```

```
  <App />
```

```
</BrowserRouter>
```



Routes, Route and Link

```
import { Routes, Route } from "react-router-dom";
```

```
<Routes>
```

```
  <Route path="/" element={<Home />} />
```

```
  <Route path="/about" element={<About />} />
```

```
</Routes>
```

```
import { Link } from "react-router-dom";
```

```
<Link to="/about">About</Link>
```





Dynamic Routing

```
const navigate = useNavigate();  
navigate("/dashboard");
```

Used after:

- Login
- Form submission
- Logout





Fetch API

Fetching, Displaying Data, Handling Errors & Loading States.

Used to:

- Get data (**GET**)
- Send data (**POST**)
- Update data (**PUT**)
- Delete data (**DELETE**)

```
fetch("https://api.example.com/data")
```





API: Application Programming Interface

An API is a **bridge** that allows two software systems **to communicate** with each other.

Think of it like a waiter in a restaurant:

- You (Frontend).
- Kitchen (Backend / Database).
- Waiter (API).





How API handles request?

- Browser sends HTTP request. (Submit Login credentials)
- Server processes request
- Server sends JSON response
- JavaScript receives data
- UI updates without reload



GET Request Example

```
useEffect(() => {  
  const fetchData = async () => {  
    try {  
      const res = await fetch("https://jsonplaceholder.typicode.com/posts");  
      const data = await res.json();  
      setPosts(data);  
    } catch (error) {  
      console.log(error);  
    }  
  };  
  fetchData();  
}, []);
```



Comparison of JS Libraries: React vs Angular vs Vue

React:

UI Library (not full framework)

Component-based architecture

Declarative UI

One-way data flow

“Learn once, write anywhere”

Core Features

Virtual DOM (fast updates)

JSX syntax

Reusable components

Hooks (useState, useEffect)

Large ecosystem (Router, Redux)



Comparison of JS Libraries: React vs Angular vs Vue

Angular:

Complete front-end framework

Opinionated & structured

Built for enterprise apps

MVC-based architecture

TypeScript-first approach

Core Features

Two-way data binding

Dependency Injection

Built-in routing

RxJS for reactive programming

CLI for project scaffolding



Comparison of JS Libraries: React vs Angular vs Vue

Vue.js

Progressive framework

Easy to integrate

Component-based

Reactive data binding

Simplicity first

Core Features:

Virtual DOM

Single File Components (.vue)

Built-in directives (v-if, v-for)

Vue Router

Pinia / Vuex for state management





End of ReactJS

```
Const endOfReactJs = () => {  
    return(  
        <> Thank you </>  
    )  
}
```



